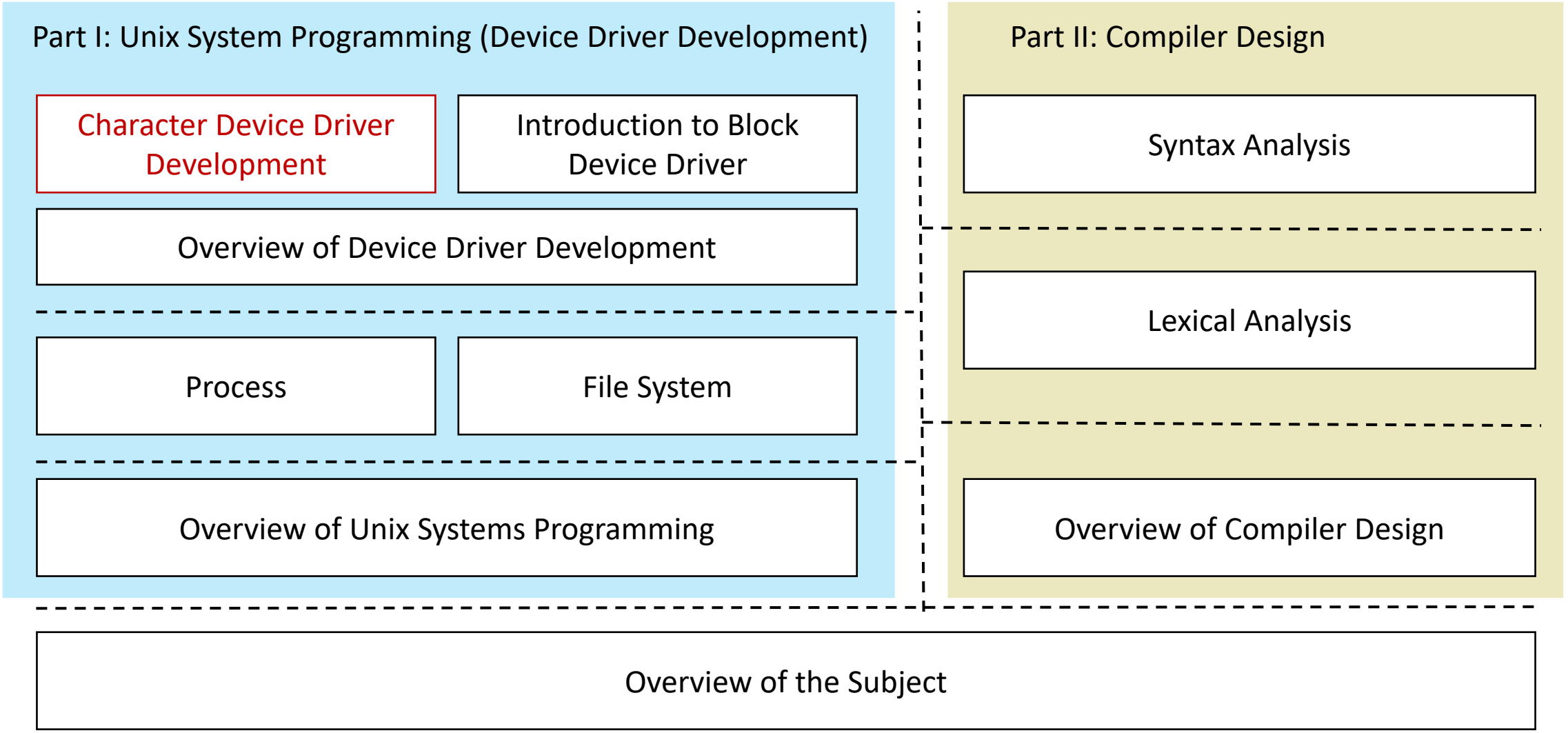
COMP3438 System Programming

Character Device Drivers (II)

Course Organization



Outline

Communication between device drivers and devices

Analysis for a simple character device driver – LED controller

- `comp_char_lab2_1.c` - control one LED (`GPIO_C5`)
- `comp_char_lab2_2.c` - control three LEDs with minor numbers

Interrupts

Semaphores

Access/Control Devices through I/O Ports

Command/data registers of devices can be accessed through I/O ports from

- memory space (memory-mapped I/O)
- separate space (I/O space)

depending on the hardware,

e.g.

ARM 32 – memory-mapped I/O

Intel 32 – I/O space (independent and different from memory space)

Input/Output Ports of ARM 2410s

ARM 2410s has 117 multifunctional input/output port pins:

Part A (GPA) - 23-pins output port
Part B (GPB) - 11-pins input/output port
Part C (GPC) - 16-pins input/output port
Part D (GPD) - 16-pins input/output port
Part E (GPE) - 16-pins input/output port
Part F (GPF) - 8-pins input/output port
Part G (GPG) - 16-pins input/output port
Part H (GPH) - 11-pins input/output port

I/O Ports in ARM 2410s

I/O ports in ARM 2410s can be configured by software to meet various system configuration and design requirements.

For out LED drivers, we will use three I/O pins that are multifunctional:

Pin	Functions	
GPC 7	Input/Output	LCDVF2
GPC 6	Input/Output	LCDVF1
GPC 5	Input/Output	LCDVF0

Multiplexed Ports

As most pins in ARM 2410s are multiplexed, we need to determine which function is selected before using it.

Usually there are three different registers for a port:

Port Configuration Register – define which function

Port Data Register

- If configured as output ports, data can be written to
- If configured as input ports, data can be read from

Port Pull-up Register – enable/disable work without pin's function setting (if we want to make pin's setting function, this should be set up as disable, i.e. set the value to 1).

Registers for Port C (GPC) in ARM 2410s

For Port C, the three corresponding registers are

`GPCCON` (configuration register), address: `0x56000020`

`GPCDAT` (data register), address: `0x56000024`

`GPCUP` (pull-up register), address: `0x56000028`

I/O Setting by Macros in the System

Instead of accessing memory addresses for I/O setting, we can use a set of macros/functions provided by the ARM 2410s architecture dependent kernel source code for I/O setting. For example,

We can configure Pin C5 as output with pull-up disabled by:

```
set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | GPIO_C5);
```

We can output one big "1" to Pin C5 by

```
write_gpio_bit(GPIO_C5, 1);
```

These macros/functions are defined in the header file:

```
kernel-2410s/include/asm/arch/S3C2410s.h
```

Our LED Experiment

In our experiment, when we plug an LED board into the extension slot in a development set, the three pins, `GPIO_C5`, `GPIO_C6`, and `GPIO_C7`, are connected to three LEDs on the LED board.

Our LED driver is to control on/off of the LEDs by writing `0`/`1` to three pins (`GPIO_C5`/`6`/`7`).

Two Steps to Control Devices in Drivers

Step 1: In the initialization of a device driver, configure I/O ports based on functions needed.

Step 2: In the implementation for `read`/`write`/`ioctl` functions of a device driver, read/write from/to the corresponding ports.

For example, in our LED driver,

in initialization, we configure the pin by

```
set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | GPIO_C5);
```

in write, we control LED by writing to `GPIO_C5` by

```
write_gpio_bit(GPIO_C5, 1);
```

Initialization in the LED Driver (comp_char_lab2_1.c)

```
static int __init zili_demo_char_led_init(void){
    int ret;
    set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | LED1);
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialization (major number -- %d).\n", major);
    return 0;
}
```

Control in the LED Driver (comp_char_lab2_1.c)

```
static ssize_t zili_demo_char_led_write(struct file *fp, const char *buf,
    size_t count, loff_t *position){
    char led_status;
    copy_from_user(&led_status, buf, sizeof(led_status));
    printk("write: led = 0x%x, count = %d.\n", led_status, count);
    if (led_status > '0')
        write_gpio_bit(LED1, 1);
    else
        write_gpio_bit(LED1, 0);
    return sizeof(led_status);
}
```

Application Program

Suppose the major number is 251. Then create a file with the major number:

```
mknod /dev/lab2 c 251 0
```

Application Program

```
#include <linux/types.h>
#include <linux/stat.h>
#include <linux/fcntl.h>
main(){
    int fd;
    char input;
    fd = open("/dev/lab2", O_WRONLY);
    input = '1';
    write(fd, &input, 1);
}
```

Minor Number

By assigning different minor numbers, we can use one device driver to control more than one devices.

An example can be found in the file `comp_309_char_lab2_2.c`

open() in comp_char_lab2_2.c

```
#define LED1 GPIO_C5
#define LED2 GPIO_C6
#define LED3 GPIO_C7
static unsigned int LED[] = {LED1, LED2, LED3};
static int major, minor;
static int zili_demo_char_led_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    minor = MINOR(inode->i_rdev);
    printk("Device " DEVICE_NAME " open (minor = %d).\n", minor);
    return 0;
}
```

open() in comp_char_lab2_2.c

```
#define LED1 GPIO_C5
#define LED2 GPIO_C6
#define LED3 GPIO_C7
static unsigned int LED[] = {LED1, LED2, LED3};
static int major, minor;
static int zili_demo_char_led_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    minor = MINOR(inode->i_rdev);
    printk("Device " DEVICE_NAME " open (minor = %d).\n", minor);
    return 0;
}
```

When `open` is called, the minor number is obtained by "`MINOR(inode->i_rdev)`"

write() in comp_char_lab2_2.c

```
#define LED1 GPIO_C5
#define LED2 GPIO_C6
#define LED3 GPIO_C7
static unsigned int LED[] = {LED1, LED2, LED3};
static int major, minor;
static int zili_demo_char_led_write(struct file *fp, const char *buf,
    size_t count, loff_t *pos){
    char led_status;
    copy_from_user(&led_status, buf, sizeof(led_status));
    printk("write: led = 0x%x, count = %d.\n", led_status, count);
    if (led_status > '0')
        write_gpio_bit(LED[minor], 1);
    else
        write_gpio_bit(LED[minor], 0);
    return sizeof(led_status);
}
```

write() in comp_char_lab2_2.c

```
#define LED1 GPIO_C5
#define LED2 GPIO_C6
#define LED3 GPIO_C7
static unsigned int LED[] = {LED1, LED2, LED3};
static int major, minor;
static int zili_demo_char_led_write(struct file *fp, const char *buf,
    size_t count, loff_t *pos){
    char led_status;
    copy_from_user(&led_status, buf, sizeof(led_status));
    printk("write: led = 0x%x, count = %d.\n", led_status, count);
    if (led_status > '0')
        write_gpio_bit(LED[minor], 1);
    else
        write_gpio_bit(LED[minor], 0);
    return sizeof(led_status);
}
```

Write a particular LED based on the minor number (one device driver can differentiate three LEDs)

Initialization in comp_char_lab2_2.c

```
static int __init zili_demo_char_led_init(void){
    int ret;
    set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | LED1);
    set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | LED2);
    set_gpio_ctrl(GPIO_MODE_OUT | GPIO_PULLUP_DIS | LED3);
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
```

Create Files with Different Minor Numbers

Suppose that the major number is 251 (after the driver is loaded into the kernel).

Create different files with different minor number but with the same major number:

```
mknod /dev/lab2_0 c 251 0
mknod /dev/lab2_1 c 251 1
mknod /dev/lab2_2 c 251 2
```

Create Files with Different Minor Numbers

Suppose that the major number is 251 (after the driver is loaded into the kernel).

Create different files with different minor number but with the same major number:

```
mknod /dev/lab2_0 c 251 0
mknod /dev/lab2_1 c 251 1
mknod /dev/lab2_2 c 251 2
```

Create Files with Different Minor Numbers

Suppose that the major number is 251 (after the driver is loaded into the kernel).

Create different files with different minor number but with the same major number:

```
mknod /dev/lab2_0 c 251 0
mknod /dev/lab2_1 c 251 1
mknod /dev/lab2_2 c 251 2
```

Application Program

```
Main(){
    int fd;
    char input;
    fd = open("/dev/lab2_0", O_WRONLY);
    input = '0';
    write(fd, &input, 1);
    sleep(2);
    input = '1';
    write(fd, &input, 1);
    close(fd);
    fd = open("/dev/lab2_1", O_WRONLY);
    input = '0';
    write(fd, &input, 1);
    sleep(2);
    input = '1';
    write(fd, &input, 1);
    close(fd);
    fd = open("/dev/lab2_2", O_WRONLY);
    input = '0';
    write(fd, &input, 1);
    sleep(2);
    input = '1';
    write(fd, &input, 1);
    close(fd);
}
```

Interrupts

(Peripheral) devices usually have to deal with the external world and processing in devices may be very slow compared to the CPU.

To make devices and CPU work in parallel, we can use "interrupts".

An interrupt is an asynchronous signal from hardware indicating the need for attention, or a synchronous event in software indicating the need for a change in execution.

In Linux, we need to register an interrupt and connect it with interrupt handler program.

```
int request_irq(unsigned int irq,
    void (*handler)(int, void *, struct pt_regs *),
    unsigned long flags, const char * dev_name, void ** dev_id);
```

Race Conditions and Semaphores

Since an interrupt can happen at any time, it can cause the interrupt handler program to be executed in the middle of another program execution. The two programs may be accessing a piece of the same resource. Thus, race conditions must be handled.

The most common ways of protecting data from concurrent accesses are:

- Use spinlocks to enforce mutual exclusion
- Use a circular buffer and avoiding shared variables
- Use lock variables that are atomically incremented and decremented
- Note: use of semaphores should be avoided in driver code (in fact, avoided in any kernel code), why?

Summary

Devices can be controlled through I/O ports or mapped memory addresses (depending on the hardware)

Data and configuration manipulations on I/O ports or mapped memory addresses

- Configuration Register
- Data Register
- Pull-up Register

Two LED drivers are introduced to show how to write a simple char driver for LEDs